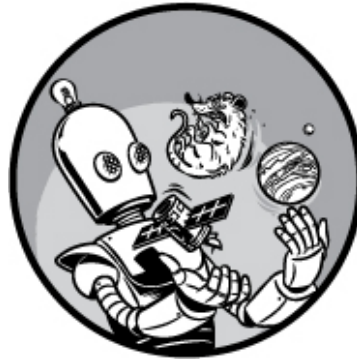


13

SIMULATING AN ALIEN VOLCANO



Quick! Name the most volcanically active body in the solar system! If you thought Earth, then you'd be mistaken—it's Io ("EYE-oh"), one of Jupiter's four Galilean moons.

The first evidence of volcanism on Io came in 1979, when *Voyager 1* made its famous fly-by of the Jovian system. But the spectacular photographs it recorded were no real surprise. Astrophysicist Stan Peale and two coauthors had already published this result based on models of Io's interior.

Computer modeling is a powerful tool for understanding nature and making predictions. Here's the general workflow:

1. Gather data.
2. Analyze, interpret, and integrate the data.
3. Generate numerical equations that explain the data.
4. Construct a computer model that best "fits" the data.
5. Use the model to make predictions and investigate error ranges.

Applications of computer modeling are far-reaching and include such areas as wildlife management, weather forecasts, climate predictions, hydrocarbon production, and black-hole simulations.

In this chapter, you'll use a Python package called `pygame`—normally used to create games—to simulate one of Io's volcanoes. You'll also experiment with different types of *ejecta* (erupted particles) and compare their simulated behaviors to a photo of Io's mammoth Tvashtar plume.

Project #21: The Plumes of Io

Tidal heating is responsible for the volcanism on Io. As its elliptical orbit carries it through the gravity fields of Jupiter and its sister moons, Io experiences variations in tidal pull. Its surface flexes up and down by as much as 100 meters, resulting in significant friction-related heating and melting in its interior. Hot magma migrates to the surface and forms great lava lakes from which degassing sulfur (S_2) and sulfur dioxide (SO_2) are sprayed skyward with speeds of 1 km/s. Because of Io's low gravity and lack of atmosphere, these plumes of gas can reach heights of hundreds of kilometers (see the Tvashtar plume in **Figure 13-1a**).

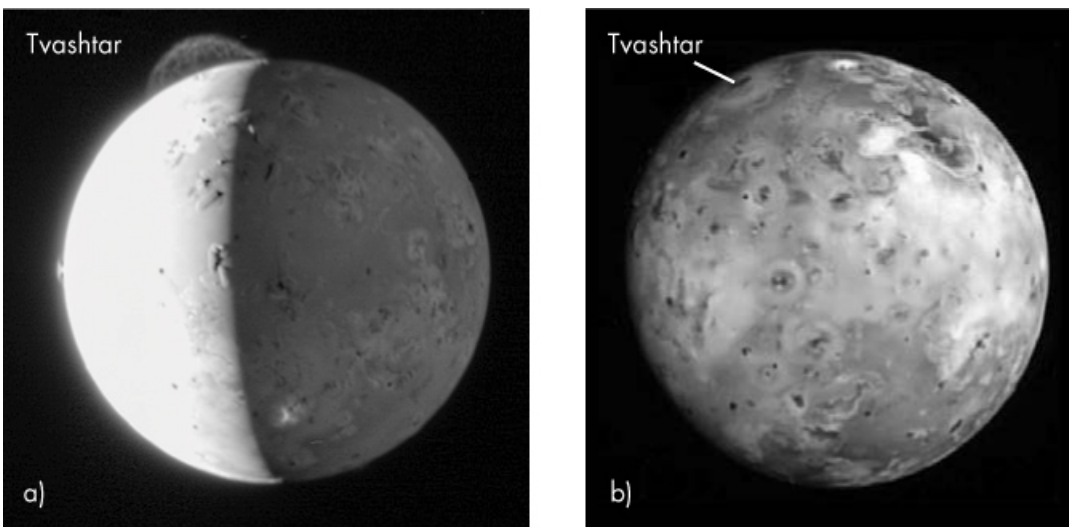


Figure 13-1: a) Io with the 330 km Tvashtar plume at the top and the shorter Prometheus plume at the 9 o'clock position; b) Io with volcanic ring deposits (NASA images)

The plumes' pleasing umbrella shape is produced as gas and dust shoot upward, then fall back in every direction. The resulting surface deposits form concentric rings of red, green, black, and yellow. If **Figure 13-1b** were in color, it would look a bit like a moldy pepperoni pizza.

A Slice of pygame

The `pygame` package is a cross-platform set of Python modules that are typically used to program 2D arcade-style video games. It supports graphics, animations, sound effects, music, and multiple input devices, such as the keyboard and mouse. Learning `pygame` is more than just a fun way to learn programming. Arcade-style games have seen a resurgence in popularity thanks to the proliferation of smartphones and tablets, and mobile gaming now earns almost as much as console and PC gaming combined.

The `pygame` package uses the *Simple DirectMedia Library (SDL)*, which is an *application programming interface (API)*. APIs are reusable code libraries that make handling graphics fairly easy, allowing you to focus on game design while using a high-level language like Python. The Microsoft *DirectX* API is used for creating games and multimedia apps for the Windows platform. For working across multiple platforms, two open source libraries are SDL, mainly for 2D work, and *OpenGL (Open Graphics Library)* for 3D applications. As mentioned, you'll be working with SDL, which officially supports Windows, macOS, Linux, iOS, and Android.

The `pygame` package also uses object-oriented programming (OOP). If you are unfamiliar with OOP or need a refresher, see “**A Brief Introduction to Object-Oriented Programming**” on **page 223**. Additionally, introductory books on Python often include a section on `pygame`, and there are whole books written on the package (see “**Further Reading**” on **page 281** for some examples).

Before you can continue, you'll need to install `pygame` onto your system. Instructions for installing a free copy on your preferred platform are available at

<http://pygame.org/wiki/GettingStarted#Pygame%20Installation>.

Video tutorials on how to install `pygame` are also available online. To ensure a video is appropriate to your situation, be sure to check the date of the video, the platform discussed, and the versions of `pygame` and Python in use. You can find additional instructions for Mac users with older Python installs at **<http://brysonpayne.com/2015/01/10/setting-up-pygame-on-a-mac/>**.

THE OBJECTIVE

Use `pygame` to build a 2D, gravity-based simulation of Io's Tvashtar volcanic plume. Calibrate the plume dimensions using a NASA image. Use multiple particle types in the plume, trace the particles' flight paths and allow the eruption to run automatically until stopped.

The Strategy

Building a comprehensive, full-physics simulation of one of Io's plumes is best accomplished with a supercomputer. Since you probably don't have one of those and since your goal is to make a cool `pygame` display, you're going to cheat by reverse-engineering the parameters you need to make SO_2 fit the Tvashtar plume. Remember, cheating is the gift humans give ourselves; it's what separates us from the animals—except for the cheetahs!

Since the composition of Io's plumes is already known, you're going to calibrate your gravity field to SO_2 and sulfur (S_2) gas, which conveniently have the same atomic weight. When the flight paths of these particles match the dimensions of the Tvashtar plume in the NASA photograph, you will scale the velocity of other ejected particles, based on the difference in atomic weight between the new particle and SO_2 , to see how particle type affects plume dimensions. Lighter particles will be ejected higher and vice versa.

Using a Game Sketch to Plan

I suggest you begin any `pygame` project with a sketch of how the game should look and how the action should progress. Even the simplest arcade games can become complicated, and a sketch will help you manage the complexity. Among the many things you must consider in a typical game are player actions, scorekeeping, messages and instructions, game entities and their interactions (such as collisions, sound effects, and music), and game-ending conditions.

Sketching the game—or in this case, the simulation—is best done on a whiteboard, either real or digital. My layout for the Io volcano simulator

is shown in **Figure 13-2**.

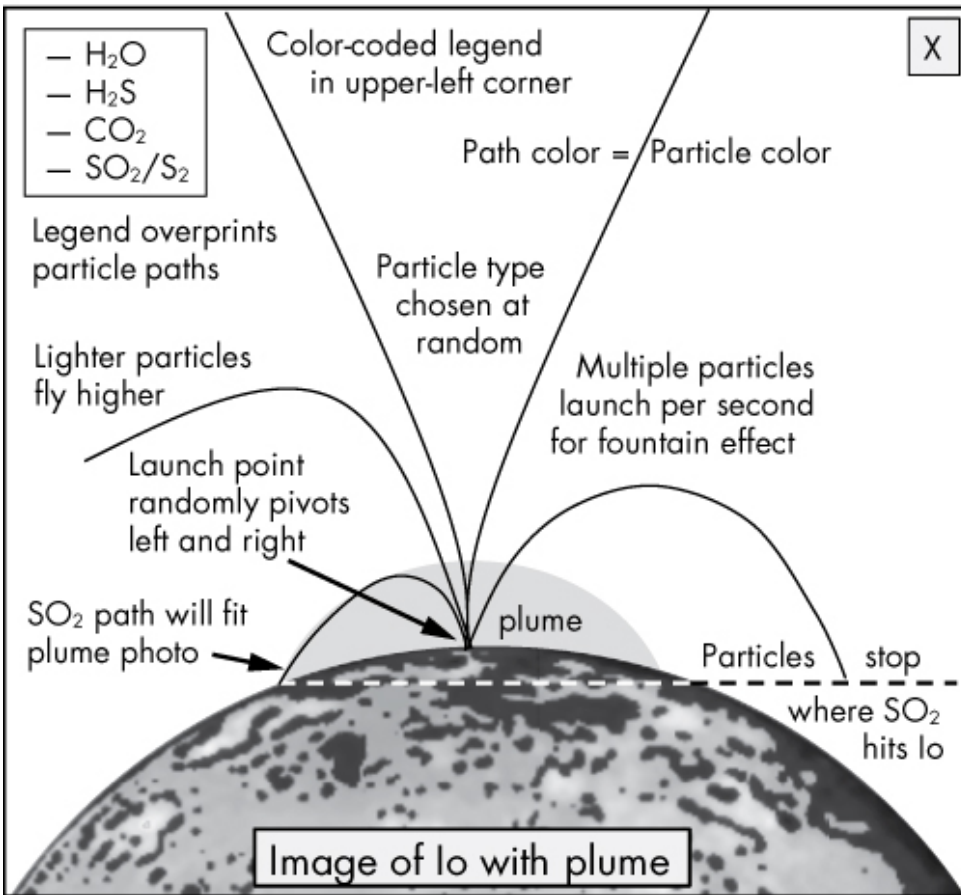


Figure 13-2: Game sketch of Io volcano simulator

The sketch in **Figure 13-2** contains guidelines and key behaviors for the volcano simulator:

- **No direct player interactions.** You'll control the simulation by editing the Python code, not through the mouse or keyboard.
- **The background will be a NASA image of a plume.** To calibrate the simulation to SO₂/S₂ particles, you'll need a backdrop of the actual Tvashtar plume.
- **The launch point pivots.** The particles should be ejected from the central base of the plume image and should spray out over a range of angles, rather than just straight up.
- **Particles are chosen at random.** The program will choose the type of particle at random. Each particle will have a unique color that distinguishes it from others.
- **The particle flight path should be visible and persistent.** The flight of each particle should be recorded as a line that remains visible

throughout the simulation, and the line color should match the particle color.

- **A color-coded legend lists the particle types.** The program should post a legend with the particle names in the upper-left corner of the screen. The font colors should match the particle colors, and the legend should print on top of particle paths so that it's always visible.
- **Particle motion should stop at the level at which SO₂ particles intersect the surface of Io.** The simulation is tuned to the behavior of SO₂, so falling particles should stop at the appropriate location for an SO₂ plume.
- **There are no sound effects.** In space, no one can hear you scream.

Once you complete your diagram, you can begin picking parts from it and listing them in logical order; this breaks the plan into a series of manageable steps. For example, you'll need to find and prepare an appropriate background image, decide which particles you want to simulate and look up their atomic weights, locate a launch point, calibrate SO₂ behavior to the plume image, and so on. You're still writing pseudocode, but a game sketch makes it a much more enjoyable process!

Planning the Particle Class

Since this simulation is based on *particles*, it makes sense to have an OOP `Particle` class to serve as a blueprint for multiple particle types. The class should support the random generation of particle types, and constants and other attributes common to all particles can be stored as *class attributes*. These are attributes assigned at the same indentation level as methods. The `Particle` class should also contain methods that allow instances of the class to be ejected, affected by gravity, made visible, and destroyed when they move beyond the boundaries of the simulation.

The attributes and methods used in the class are shown in **Tables 13-1** and **13-2**, respectively. Class attributes—that is, those shared by all instances of the class—are shown in italics; otherwise, instance attributes are shown.

Table 13-1: Attributes of the `Particle` Class (*Italics* = Class Attribute)

Attributes	Attribute description
<i>gases_colors</i>	Dictionary of available particle types and their colors
<i>VENT_LOCATION_XY</i>	x- and y-location of mouth of Tvashtar volcano in image
<i>IO_SURFACE_Y</i>	Io surface y-value at SO ₂ plume margin in image
<i>VELOCITY_SO2</i>	Speed (pixels per frame) of SO ₂ particle
<i>GRAVITY</i>	Acceleration of gravity in pixels per frame
<i>vel_scalar</i>	Dictionary of ratios of SO ₂ /particle atomic weights
screen	The game screen
background	A NASA image of the Tvashtar plume
image	A square <code>pygame</code> surface representing a particle
rect	A rectangular object used to get surface dimensions
gas	The type of an individual particle (SO ₂ , CO ₂ , etc.)
color	Color of an individual particle type
vel	Particle's velocity relative to SO ₂ velocity
x	Particle's x-location

Attributes	Attribute description
y	Particle's y-location
dx	Particle's delta-x
dy	Particle's delta-y

Table 13-2: Methods of the `Particle` Class

Method	Method description
<code>__init__()</code>	Initializes and sets up parameters for randomly selected particle type
<code>vector()</code>	Randomly selects ejection orientation and calculates motion vector (dx and dy)
<code>update()</code>	Adjusts particle trajectory for gravity, draws path behind particle, and destroys particles that move beyond the boundaries of the simulation

I will explain each of these attributes and methods in more detail in the next section.

The Code

The *tvastar.py* code will generate a `pygame`-based simulation of Io's Tvashtar plume. You will also need the background image, *tvastar_plume.gif*. Download both files from <https://www.nostarch.com/impracticalpython/> and keep them in the same folder.

Importing Modules, Initiating pygame, and Defining Colors

Begin with several set-up steps, such as choosing colors, as shown in **Listing 13-1**.

tvashtar.py, part 1

```
❶ import sys
    import math
    import random
    import pygame as pg

❷ pg.init() # initialize pygame

❸ # define color table
    BLACK = (0, 0, 0)
    WHITE = (255, 255, 255)
    LT_GRAY = (180, 180, 180)
    GRAY = (120, 120, 120)
    DK_GRAY = (80, 80, 80)
```

Listing 13-1: Imports modules, initiates `pygame`, and defines a table of colors

Start with some familiar `import`s and one for `pygame` ❶. Next, call the `pygame.init()` function. This initializes the `pygame` module and starts up all the underlying parts that let it use sounds, check keyboard input, run graphics, and so on ❷. Note that `pygame` can be initialized from multiple places, such as the first line in a `main()` function:

```
def main():
    pg.init()
```

or at the end of the program when `main()` is called in stand-alone mode:

```
if __name__ == "__main__":
    pg.init()
    main()
```

Stop and assign some color variables using the RGB color model ③. This model mixes red, green, and blue, where each color consists of values from 0 to 255. If you do an online search for “RGB color codes,” you can find the numerical codes for millions of colors. But since the NASA image you’ll calibrate against is in grayscale, stick with black, white, and shades of gray. Defining this table now will allow you to simply enter a name when `pygame` needs a color defined later.

Defining the Particle Class

Listing 13-2 defines the `Particle` class and its initializer method. You’ll use these to instantiate a particle object. The particle’s key attributes, such as type, velocity, color, and so on, are established with the initializer method.

tvashtar.py, part 2

```
❶ class Particle(pg.sprite.Sprite):
    """Builds ejecta particles for volcano simulation."""

    ❷ gases_colors = {'SO2': LT_GRAY, 'CO2': GRAY, 'H2S': DK_GRAY, 'H2O': WHITE}

    ❸ VENT_LOCATION_XY = (320, 300)
    IO_SURFACE_Y = 308
    GRAVITY = 0.5 # pixels-per-frame; added to dy each game loop
    VELOCITY_SO2 = 8 # pixels-per-frame

    # scalars (SO2 atomic weight/particle atomic weight) used for velocity
    ❹ vel_scalar = {'SO2': 1, 'CO2': 1.45, 'H2S': 1.9, 'H2O': 3.6}

    ❺ def __init__(self, screen, background):
        super().__init__()
        self.screen = screen
        self.background = background

        ❻ self.image = pg.Surface((4, 4))
        self.rect = self.image.get_rect()

        ❼ self.gas = random.choice(list(Particle.gases_colors.keys()))
```

```
        self.color = Particle.gases_colors[self.gas]
    ❸ self.vel = Particle.VELOCITY_SO2 * Particle.vel_scalar[self.gas]
    ❹ self.x, self.y = Particle.VENT_LOCATION_XY
    ❺ self.vector()
```

Listing 13-2: Defines the `Particle` class and the `Particle` initializer method

Define a class called `Particle` to represent *any* molecule of gas that might form the volcanic plume ❶. The *ancestor* for this class, shown in parentheses, will be a `Sprite` class. This means that the `Particle` class is derived from a built-in `pygame` type called `Sprite`. Sprites are just 2D bitmaps that represent discrete game objects, like missiles or asteroids. You *inherit* from the `Sprite` class—that is, add its attributes and methods to your new class—by passing `pg.sprite.Sprite` to your `Particle` class, just as you would pass an argument to a function.

Assign properties common to all particles as class attributes. The first is a dictionary that maps particle types to a color, so that you can differentiate the particles during the simulation ❷. These colors will be used for the particle, its path, and its name in the legend.

Now, assign four constants, `VENT_LOCATION_XY`, `IO_SURFACE_Y`, `GRAVITY`, and `VELOCITY_SO2` ❸. The first constant is the x- and y-coordinates for the mouth of the Tvashtar volcano in the image, which will represent the “launch point” for all the particles (see **Figure 13-3**). I initially guessed at these values and then fine-tuned them when the simulation was up and running.

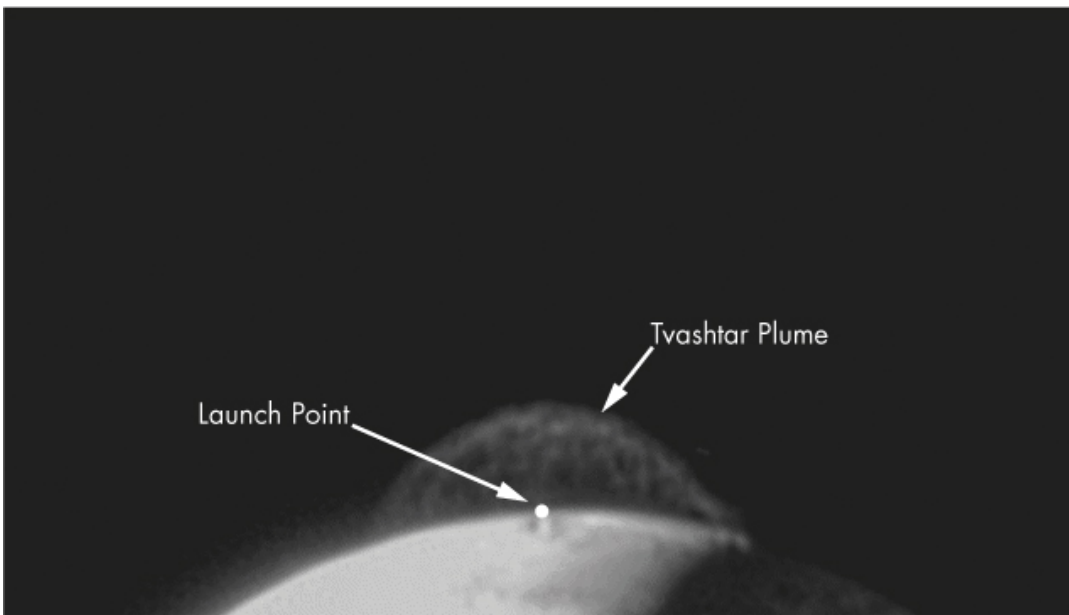


Figure 13-3: Background image for the simulation with the particle launch point annotated

The second constant is the y-value for the highest point on Io's surface (in the image) that intersects with the outer edge of the SO₂ plume (see **Figure 13-2**). You will stop all the falling particles at this y-value, so the view will be optimized for SO₂.

The third constant represents the acceleration of gravity, which on Earth is 9.86 m/s² and on Io is 1.796 m/s². But you're dealing with pixels and frames here, not real-world units, so you need to experiment to find a value that looks good at the scale of your game/simulation. My choice of 0.5 was arbitrary, though guided by knowledge of what works well in arcade games.

The fourth constant is the speed, in pixels per frame, of an SO₂ particle when it is ejected. Remember, the plumes are composed primarily of SO₂, so you want to use parameters that will allow the SO₂ particles to “fit” the image of the Tvashtar plume, then adjust the speed of the remaining particles *relative* to SO₂. Neither the GRAVITY nor the VELOCITY_SO2 value is unique. Had I chosen a larger value for GRAVITY, I would've needed to increase VELOCITY_SO2 so that the SO₂ particles still “fill out” the area of the plume in the NASA image.

Next, build a dictionary of scalars for particle velocity ④. For each particle, dividing the atomic weight of SO₂ (64) by the atomic weight of the

particle gives you the scalar. Since SO_2 is the reference particle, its scalar is 1. Later, to get the velocity of non- SO_2 particles, you will multiply the `VELOCITY_SO2` constant by the scalar. As you can see, all of the other particles are lighter than SO_2 and should produce a larger plume.

Define a constructor method for the particle object ⑤. You'll need a `self` parameter and `screen` to draw on and check the simulation's boundaries, and you'll need a `background`, which will be an image of the Tvashtar plume. You'll assign `screen` and `background` later in the `main()` function, defined near the end of the program. Note that while I'm using one-line docstrings in this book for brevity, you would want to include these types of parameters in the class docstring. For more guidelines on class docstrings, see <https://www.python.org/dev/peps/pep-0257/>.

Inside the `__init__()` method, immediately invoke the initialization method for the built-in `Sprite` class using `super`. This will initialize the sprite and establish the `rect` and `image` attributes it needs. With `super`, you don't need to refer to the base class (`Sprite`) explicitly. For more on `super`, visit the docs at <https://docs.python.org/3/library/functions.html#super>.

Next, let the particle (`self`) know that it will be using the `screen` and `background` variables by assigning them to attributes.

The images and drawings are placed on a rectangular surface by `pygame`. In fact, the `Surface` object is the heart and soul of `pygame`; even the `screen` attribute represents an instance of `Surface`. Assign the particle image to a `Surface` object and make it a square with 4-pixel-length sides ⑥.

Next, you need to get a `rect` object for your image surface. This is basically a rectangle *associated* with the `Surface` object, which `pygame` needs to determine the dimensions and position of your `Surface` object.

Choose a particle (`gas`) type by randomly choosing from the keys in the `gases_colors` dictionary ⑦. Note that you turn it into a list to do this. Since it's possible to assign an instance attribute named `gases_colors` from within the `__init__()` method, include the class name—rather than `self`—to ensure the *class* attribute is being referenced.

Once you have a type, you can use it as the key in the dictionaries you built earlier to access things like colors and scalars. Start by getting the correct color for the chosen particle, then get its `vel_scalar` value and use that to determine the particle's velocity ❸.

The particle object will be instantiated in the mouth of the volcano, so get its initial x- and y-location by unpacking the `VENT_LOCATION_XY` tuple ❹. Finish by calling the `vector()` method, which will calculate the particle's motion vector ❺.

Ejecting a Particle

Listing 13-3 defines the `vector()` method, which determines a particle's launch orientation and calculates its initial delta-x and delta-y vector components.

tvashtar.py, part 3

```
❶ def vector(self):
    """Calculate particle vector at launch."""
    ❷ orient = random.uniform(60, 120) # 90 is vertical
    ❸ radians = math.radians(orient)
    ❹ self.dx = self.vel * math.cos(radians)
    self.dy = -self.vel * math.sin(radians)
```

Listing 13-3: Defines the `vector()` method of the `Particle` class

The `vector()` method ❶ calculates the motion vector for a particle. Start by choosing a launch direction for the particle and assigning it to an `orient` variable ❷. Because explosive volcanic eruptions blast material in multiple directions rather than straight up, choose the direction at random, using a range that is 30 degrees to either side of 90 degrees, where 90 represents a vertical launch.

The range for the `orient` variable was chosen through trial and error. This parameter, along with the `VELOCITY_SO2` and `GRAVITY` constants, represents the “knobs” you can turn to calibrate the behavior of the `SO2` particle to the

plume image. After you've adjusted the constants so that the particle's maximum height corresponds to the apex of the plume, you can adjust the range of angles so that the SO₂ particles reach—but don't exceed—the lateral limits of the plume (see **Figure 13-4**).



Figure 13-4: Calibrating the `orient` variable to the Tvashtar plume

The `math` module uses *radians* rather than degrees, so convert `orient` to radians ③. A radian is a standard unit of angular measurement equal to the angle made when the radius is wrapped around a circle (see the left-hand side of **Figure 13-5**). One radian is slightly less than 57.3 degrees. The right-hand side of **Figure 13-5** is a comparison of radians and degrees for some common angles. To convert degrees to radians, you can either multiply the degrees by π and divide by 180—like a chump—or just use the `math` module!

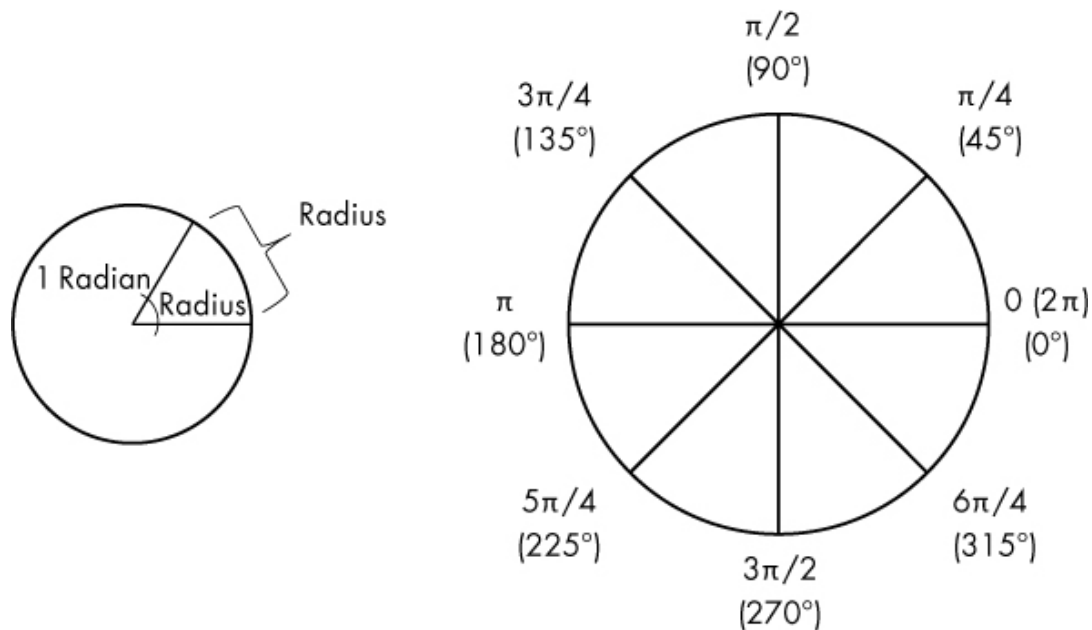
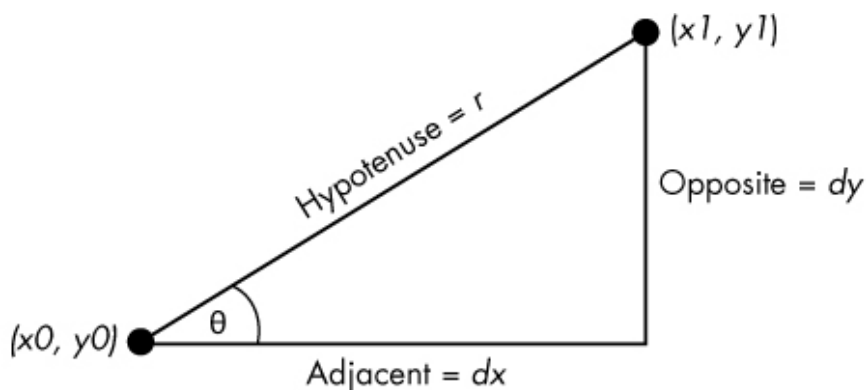


Figure 13-5: Definition of a radian (left) and common angles in radians and degrees (right)

Objects move in `pygame` by increments of `x` and `y`. The direction and speed of a particle are used to get its *delta-x* (dx) and *delta-y* (dy) vector components. These represent the difference between a particle's initial position and its position after completion of a single game loop.

You calculate vector components using trigonometry. Useful trigonometric equations are provided in **Figure 13-6**.



$$\sin(\theta) = \text{opp/hyp} = dy/r$$

$$dy = r \times \sin(\theta)$$

$$\tan(\theta) = \text{opp/adj} = dy/dx$$

$$\cos(\theta) = \text{adj/hyp} = dx/r$$

$$dx = r \times \cos(\theta)$$

$$r^2 = dx^2 + dy^2$$

$$r = \sqrt{dx^2 + dy^2}$$

Figure 13-6: Common trigonometric equations used in gaming

For the angle θ , you use the `orient` variable. The `self.vel` attribute equates to r . Knowing these two components, you can use a trigonometric equation to derive `self.dx` and `self.dy` ❷. To derive `self.dx`, multiply `self.vel` by the cosine of `orient`, and to derive `self.dy`, multiply `self.vel` by the sine of `orient`. Note that you must make `self.dy` negative, as the particles are ejected upward and y-values in `pygame` increase *downward*.

Updating the Particle and Handling Boundary Conditions

Listing 13-4 completes the `Particle` class by defining a method to update particles as they move across the screen. This includes applying the force of gravity, drawing a line to trace the particle's path, and “killing” the particle when it moves offscreen or below the surface of Io.

tvashtar.py, part 4

```
❶ def update(self):
    """Apply gravity, draw path, and handle boundary conditions."""
    ❷ self.dy += Particle.GRAVITY
    ❸ pg.draw.line(self.background, self.color, (self.x, self.y),
                  (self.x + self.dx, self.y + self.dy))
    ❹ self.x += self.dx
    self.y += self.dy

    ❺ if self.x < 0 or self.x > self.screen.get_width():
        ❻ self.kill()
    ❼ if self.y < 0 or self.y > Particle.IO_SURFACE_Y:
        self.kill()
```

Listing 13-4: Defines the `update()` method and completes the `Particle` class

Define the `update()` method, which takes `self` as an argument ❶. Apply the force of gravity by adding the `GRAVITY` class attribute to `self.dy` during each game loop ❷. Gravity is a force vector that works only in the vertical direction, so only `self.dy` is affected.

To draw a path behind the particle, use `pygame`'s `draw.line()` method, which takes the background image of Io, the color of the particle, and the coordinates for the previous and current locations of the particle as arguments ❸. To get the current location, you add the `self.dx` and `self.dy` attributes to `self.x` and `self.y`.

Next, update the particle's `self.x` and `self.y` attributes by adding `self.dx` and `self.dy`, just as you did in the `draw.line()` method ❹.

Now, check to see if the particle has passed the left or right boundaries of the screen ❺. Use a `self.x` equal to zero for the left side and get the width of the `screen` attribute for the right side. If the particle has passed off either side of the screen, use the built-in `kill()` method to remove it from all the groups that contain it ❻. As you will see later, `pygame` uses containers—called *groups*—to manage sprites, and removing a sprite from a group takes it out of play.

Repeat this process for the y-direction ❼, but for a maximum value, use the `IO_SURFACE_Y` constant of the `Particle` class, which will stop the particle near the surface of Io, where an SO_2 particle would stop (see **Figures 13-2** and **13-4**).

Defining the main() Function

Listing 13-5 defines the first part of the `main()` function that sets up the game screen, the window caption, the legend, the sprite group, and the game clock.

tvashtar.py, part 5

```
def main():
    """Set up and run game screen and loop."""
    ❶ screen = pg.display.set_mode((639, 360))
    ❷ pg.display.set_caption('Io Volcano Simulator')
    ❸ background = pg.image.load('tvashtar_plume.gif')

    # Set up color-coded legend
    ❹ legend_font = pg.font.SysFont('None', 24)
```

```
⑤ water_label = legend_font.render('--- H2O', True, WHITE, BLACK)
   h2s_label = legend_font.render('--- H2S', True, DK_GRAY, BLACK)
   co2_label = legend_font.render('--- CO2', True, GRAY, BLACK)
   so2_label = legend_font.render('--- SO2/S2', True, LT_GRAY, BLACK)

⑥ particles = pg.sprite.Group()

⑦ clock = pg.time.Clock()
```

Listing 13-5: Defines the first part of the `main()` function

The first step is to assign the `screen` variable using `pygame`'s `display.set_mode()` method ❶. The arguments are pixel dimensions; in this case, you use values slightly smaller than those for the NASA image to ensure a good fit. Note that the dimensions must be provided as a tuple, so you need to include two sets of parentheses.

Next, name your game window using `pygame`'s `display.set_caption()` method ❷, then assign the `background` variable to the NASA photo of the Tvashtar plume ❸. Use `pygame`'s `image.load()` method to create a new `Surface` object from the image. The `pygame` package supports several image formats, including PNG, JPG, and GIF. The returned `Surface` will inherit color and transparency information from the image file. Since you're importing a grayscale image here, your color choices will be limited.

Now, add some code to build the legend that will be displayed in the upper left of the screen.

Name a `legend_font` variable and use `pygame`'s `font.SysFont()` method to choose `None` at size 24 ❹. You will use this when you render the text. The `pygame` package's `font` module lets you render a new set of fonts, called TrueType fonts, onto a new `Surface` object. If you don't want to specify a font, `pygame` comes with a built-in default font, which you can access by passing `None` as the font name.

Post the particle names in order of weight, with the lightest on the top. To make a label, call `render()` on the `legend_font` object you made earlier to gen-

erate a new surface object ❸. Pass it some text, then `True` (to turn on anti-aliasing so the text looks smoother), and then the color of the particle being described. The last argument, `BLACK`, is optional and sets the background color of the label to black so the text will be legible above all the particle paths drawn on the screen. Repeat this process for the three remaining particles and add `s2` to the `so2_label`, as both gases share the same atomic weight and will behave the same in the simulation.

Now, start a sprite group named `particles` ❹. Since games typically have multiple sprites moving around the screen, `pygame` uses a container—the sprite group—to manage them. In fact, you *must* put sprites in a group or else they won't do anything.

Finish this section by creating a `clock` object to track and control the frame rate of the simulation ❺. A `pygame` “clock” controls how fast the game runs, based on the number of *frames per second (fps)* being displayed. You'll set this value in the next section.

Completing the `main()` Function

Listing 13-6 completes `main()` by setting the speed the simulation will run—in frames per second—and starting the `while` loop that actually runs the simulation. It also handles *events*, which occur when a user exerts control on the program using a mouse, joystick, or keyboard. Since this is a simulation and not a true game, user control is limited to closing the window. The listing ends in the global scope with the standard code for running the program as a module or in stand-alone mode.

tvashtar.py, part 6

```
❶ while True:
    ❷ clock.tick(25)
    ❸ particles.add(Particle(screen, background))
    ❹ for event in pg.event.get():
        if event.type == pg.QUIT:
            pg.quit()
            sys.exit()
```

```
    ⑤ screen.blit(background, (0, 0))
      screen.blit(water_label, (40, 20))
      screen.blit(h2s_label, (40, 40))
      screen.blit(co2_label, (40, 60))
      screen.blit(so2_label, (40, 80))

    ⑥ particles.update()
      particles.draw(screen)

    ⑦ pg.display.flip()

    ⑧ if __name__ == "__main__":
      main()
```

Listing 13-6: Starts the game clock and loop and handles events in the `main()` function

Start a `while` loop to run the simulation ❶. Then use the `clock.tick()` method to set the speed limit for the simulation ❷. Pass it 25, which sets the maximum frame rate at 25 frames per second. Feel free to increase this value if you want a more energetic volcano.

It's now time for the star of the show to make its appearance. Instantiate a particle using the `Particle` class, passing it the `screen` and `background` as arguments, and add the new particle to the `particles` sprite group ❸. With each frame, a new particle will be created at random and launched from the volcanic vent, producing a pleasing spray of particles (see **Figure 13-7**).



Figure 13-7: Startup of the simulation, with random particles being generated at the rate of 25 frames per second

Start a `for` loop to handle events ④. All the events that happened during the current frame are recorded by `pygame` and kept in an *event buffer*. Its `event.get()` method creates a list of all these events so that you can evaluate them in turn. If a `QUIT` event occurs (when a user closes the game window), the `pygame quit()` and `system exit()` methods are called to end the simulation.

To render the game objects and update the visual display, `pygame` uses a process called *blitting*. *Blit* stands for *block transfer*, which is just copying pixels from one rectangular `Surface` object onto another. By blitting the background onto the screen, you cover the screen with the Io image. With blitting, you can take the same image and draw it multiple times at different places on the screen. It can be a slow process, so game developers use clever techniques to address this deficiency, such as blitting only around areas that are currently being updated, rather than blitting the whole screen in every game loop.

To blit the background onto the screen, call the `blit()` method on the screen and pass it the required arguments for source and destination ⑤. In the first example, the `background` variable is the source, and the destination is the coordinates for where you want the *upper left-hand corner* of the background. Since the background will cover the whole screen, use

the origin point for the screen, which is `(0, 0)`. Repeat this for the legend labels, placing them in the upper-left corner of the screen.

Next, call the `update()` method on the `particles` group ❸. This method doesn't update the screen but has the sprites run their own `self.update()` methods. After this, you use the `draw()` method to blit the sprites onto the screen based on each sprite's `rect` attributes. This method needs a drawing surface, so pass it the `screen`.

The `draw()` method took care of blitting sprites, so all you need to do now is use the `flip()` method to update the actual game graphics ❹. *Flipping* is a type of double buffering where you blit everything from the `screen` object to the actual display. Flipping gets around the inherently slow process of displaying graphics, which can cause the screen to flicker, by doing the work on a behind-the-scenes rectangle and then using a version of the `blit()` method to copy to the final display

The listing ends outside the `main()` function with the code that lets the program run as a module or in stand-alone mode ❺.

Running the Simulation

Figure 13-8 shows the result of running the simulator for about a minute. The water vapor plume extends beyond the top of the window. The next highest plume is formed by hydrogen sulfide, followed by carbon dioxide, and then sulfur dioxide/sulfur (S_2) gas, which by design perfectly matches the Tvashtar plume.

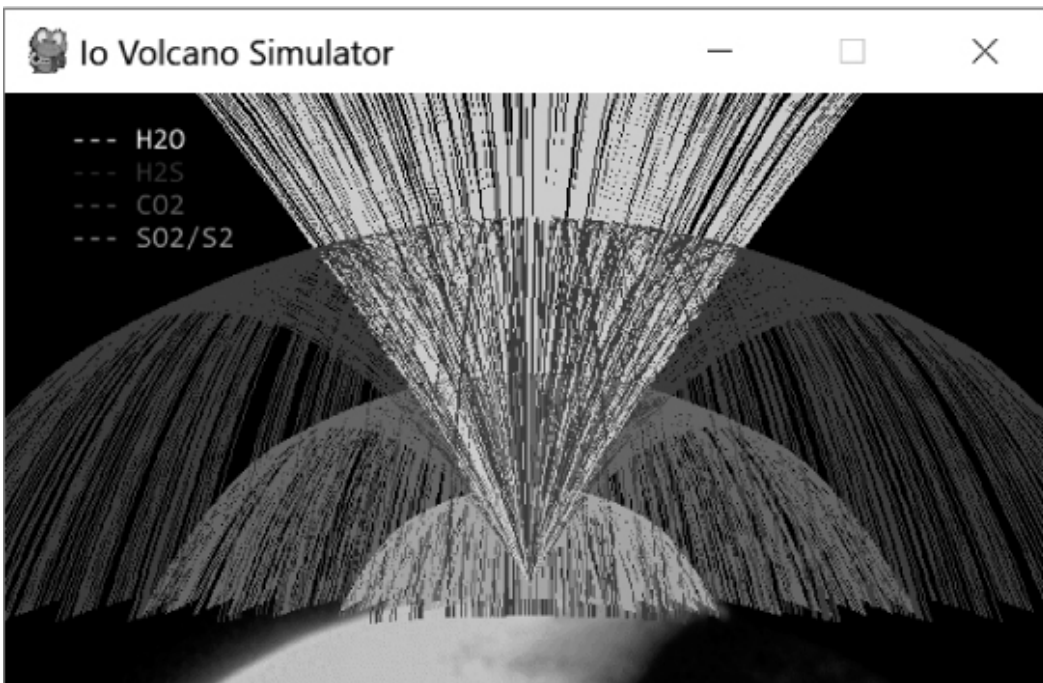


Figure 13-8: Results of running `tvastar.py` for one minute

To run the simulator with *only* SO₂, go to the `__init__` method of the `Particle` class and change the lines where you select the `gas` and `color` instance attributes:

```
self.gas = 'SO2'  
self.color = random.choice(list(Particle.gases_colors.values()))
```

By choosing a color at random, you maintain a sense of movement in the plume after all of the possible `self.orient` angles have been exhausted. And if you want to speed up or slow down the eruption, go to the `main()` function and experiment with the `clock.tick()` method’s frames-per-second parameter.

In real life, the composition of plume material was inferred with *spectroscopy*, a measurement technique that analyzes how light interacts with matter. This includes both visible and nonvisible wavelengths that are absorbed, emitted, or scattered. The “spectra of the ejecta,” together with the colors painted on the surface, provided the key evidence for sulfur-rich plumes.

Summary

In this chapter, you learned how to use the `pygame` package to simulate gravity and build an animation of an extraterrestrial volcano. In the next chapter, you'll use `pygame` to build a true arcade game with player interactions and win-lose conditions.

Further Reading

Game Programming: The L-Line, The Express Line to Learning (Wiley, 2007) by Andy Harris is a tremendously useful and thorough 570-page introduction to `pygame`.

More Python for the Absolute Beginner (Cengage Learning Course Technology, 2012) by Jonathon Harbour builds on the prequel, *Python for the Absolute Beginner*, using a (py)game-focused approach.

Invent Your Own Computer Games with Python, 4th Edition (No Starch Press, 2016) by Al Sweigart is a good introduction to both Python and game design for beginners.

An online “newbie guide” for `pygame` is available at <https://www.pygame.org/docs/tut/newbieguide.html>, and a “cheat sheet” can be found at <http://www.cogsci.rpi.edu/~destem/gamedev/pygame.pdf>.

Three-Dimensional Simulation of Gas and Dust in Io's Pele Plume, by William J. McDoniel and others, documents the simulation of Io's Pele Plume using direct Monte Carlo simulation and supercomputers at the Texas Advanced Computing Center at the University of Texas. The article is available at http://cfpl.ae.utexas.edu/wp-content/uploads/2016/01/McDoniel_PeleDust.pdf.

Practice Project: Going the Distance

You're one of King Henry's archers at the battle of Agincourt. The French are charging, and you want to strike them as far away as possible. At what angle do you hold your longbow?

If you've ever had a physics class, you probably know the answer is 45 degrees. But can you trust that pencil-necked physicist? Better run a quick

computer simulation to check it out. Copy and edit the *tvashtar.py* code to randomly fire particles at 25, 35, 45, 55, and 65 degrees. Set `self.color` to `WHITE` for 45 degrees and `GRAY` for all other angles (see **Figure 13-9**).

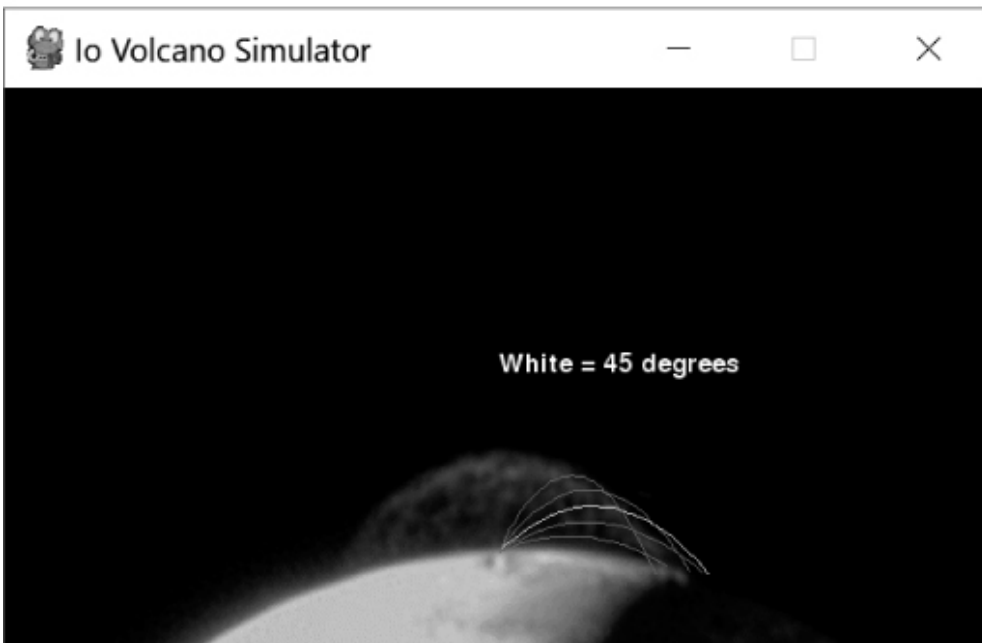


Figure 13-9: Io volcano simulator modified for ejection angles of 25, 35, 45, 55, and 65 degrees

You can find a solution, *practice_45.py*, in the appendix or for download at <https://www.nostarch.com/impracticalpython/>. Keep it in the same folder as the *tvashtar_plume.gif* file.

Challenge Projects

Continue your experiments with these challenge projects. No solutions are provided.

Shock Canopy

The visibility of Io's giant plumes is believed to be enhanced by gas condensing into dust in the *shock canopy*, the point where the gas particles reach their apex and begin falling back to the surface. Use the `self.dy` attribute to edit the path colors in a copy of the *tvashtar.py* program. Paths in the apex of the plume should be brighter than those below (see **Figure 13-10**). As with all Challenge Projects, no solution is provided.

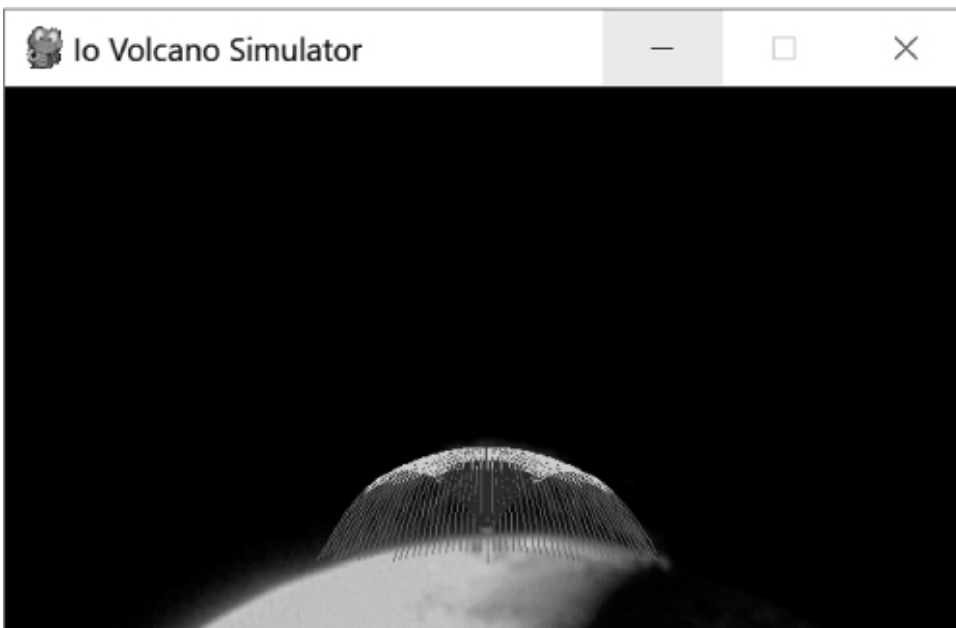


Figure 13-10: Highlighting the shock canopy using lighter path colors

The Fountainhead

Copy and edit `tvashtar.py` so that only SO_2 is simulated and particles are represented by small white circles with no trailing paths (see **Figure 13-11**).

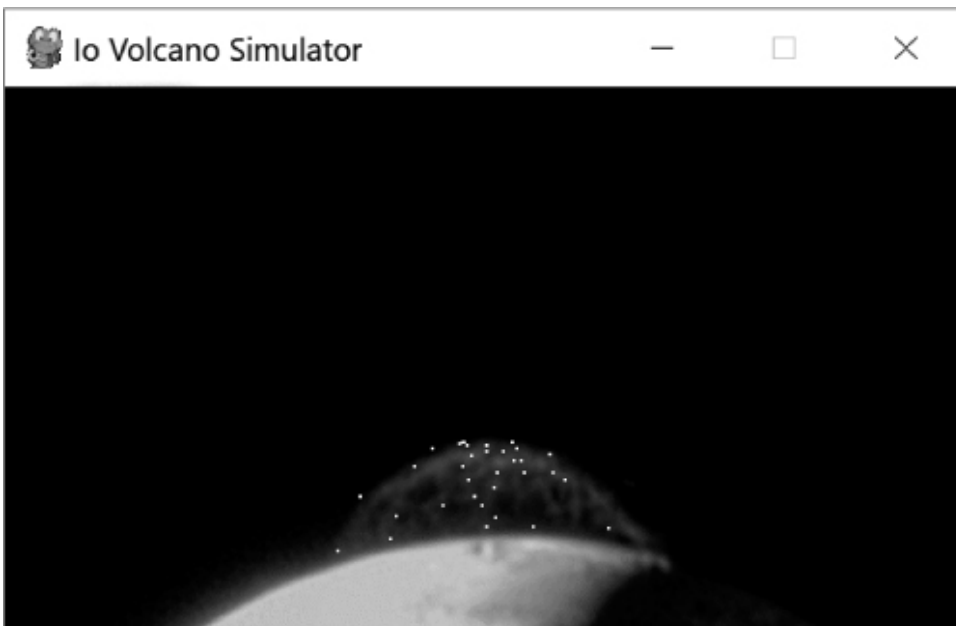


Figure 13-11: Screenshot of SO_2 simulation with circles representing individual particles

With a Bullet

If you fire a gun straight up on a planet with no atmosphere, will the bullet hit the ground with the same velocity as when it left the muzzle? Many

people struggle with this question, but you can answer it with Python. Copy and edit the *tvashtar.py* code so that it ejects a single SO₂ particle with an orientation of 90 degrees. Print the particle's `self.y` attribute and the absolute value of `self.dy` at the launch point coordinate (`y = 300`). Compare the starting and ending velocity values at this point to see whether they are the same or similar.

NOTE

Episode 50 of the TV show MythBusters addressed the myth that bullets fired into the air maintain their lethal capability when they eventually fall back down. They found that bullets fired perfectly vertical on Earth would tumble and slow on their way back down due to wind resistance. If fired slightly off-vertical, the bullets would maintain their spin and ballistic trajectory and return to Earth at lethal speed. It was the only myth that ever received all three ratings (Busted, Plausible, and Confirmed)!
